NAME:DARSHAN D

REG NO:192524117

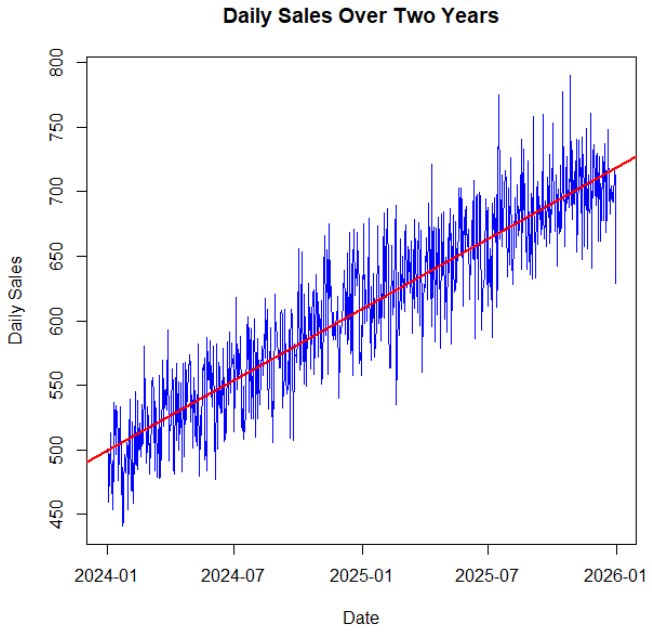
Unit IV Activity II

1. A retail chain's finance team wants to understand how daily sales have moved over the past two years before setting next year's budget. Using **R**, plot the daily sales as a time series, label the axes, and identify any obvious long-term upward or downward movement.

R program:

```
# Sample daily sales for 2 years
set.seed(10)

dates <- seq(as.Date("2024-01-01"), as.Date("2025-12-31"), by = "day")
sales <- 500 + (1:length(dates)) * 0.3 + rnorm(length(dates), 0, 30)

# Plot
plot(dates, sales,
     type = "l",
     col = "blue",
     main = "Daily Sales Over Two Years",
     xlab = "Date",
     ylab = "Daily Sales")

# Trend line
abline(lm(sales ~ as.numeric(dates)), col = "red", lwd = 2)
```

Output:



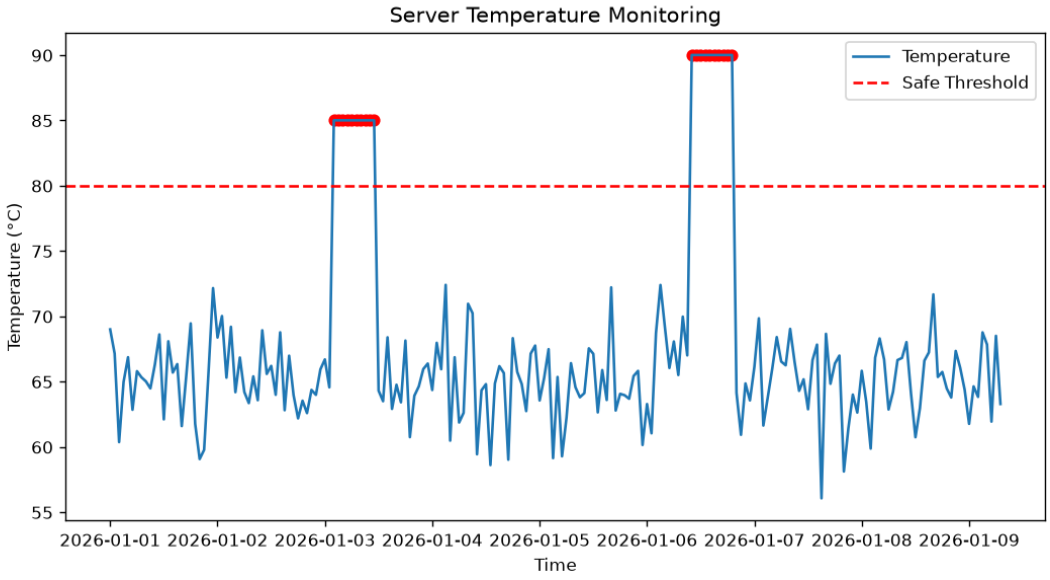
2. An IoT company monitors hourly server temperature at a data center to catch overheating risks early. Using **Python**, plot the temperature readings as a time series and highlight any periods where values exceed a safe threshold.

Python program:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt



np.random.seed(10)



time = pd.date_range("2026-01-01", periods=200, freq="h")

temperature = 65 + np.random.normal(0, 3, 200)
```

```
# Add overheating periods
temperature[50:60] = 85
temperature[130:140] = 90

threshold = 80

plt.figure(figsize=(10,5))
plt.plot(time, temperature, label="Temperature")
plt.axhline(threshold, color="red", linestyle="--", label="Safe Threshold")

plt.scatter(time[temperature > threshold],
            temperature[temperature > threshold],
            color="red")

plt.xlabel("Time")
plt.ylabel("Temperature (°C)")
plt.title("Server Temperature Monitoring")
plt.legend()
plt.show()
```

Output:



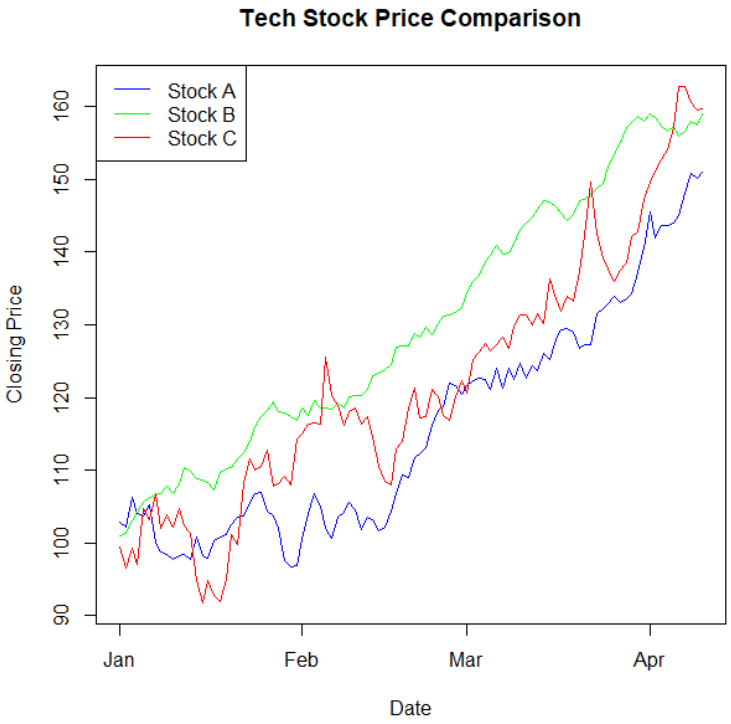
3. A hedge fund analyst is comparing the daily closing prices of three competing tech stocks to decide which one shows steadier growth. Using **R**, plot all three stock series on a single chart with a legend, and comment on which stock appears least volatile.

R program:

```
set.seed(20)


dates <- seq(as.Date("2025-01-01"), by = "day", length.out = 100)


stock_A <- 100 + cumsum(rnorm(100, 0.5, 2))

stock_B <- 100 + cumsum(rnorm(100, 0.5, 1))

stock_C <- 100 + cumsum(rnorm(100, 0.5, 3))


plot(dates, stock_A,
```

```
    type = "l",
    col = "blue",
    ylim = range(c(stock_A, stock_B, stock_C)),
    xlab = "Date",
    ylab = "Closing Price",
    main = "Tech Stock Price Comparison")

lines(dates, stock_B, col = "green")
lines(dates, stock_C, col = "red")

legend("topleft",
       legend = c("Stock A", "Stock B", "Stock C"),
       col = c("blue", "green", "red"),
       lty = 1)
```

Output:



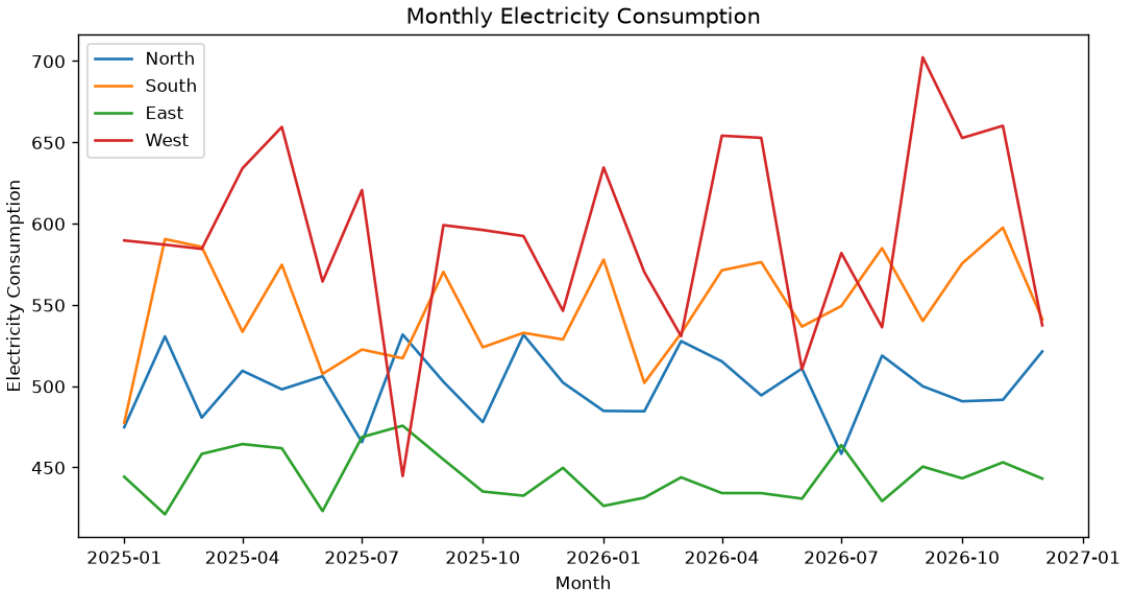
4. A city's energy board wants to compare monthly electricity consumption across four different neighborhoods to plan targeted conservation campaigns. Using **Python**, plot the four time series together and identify the neighborhood with the most erratic consumption pattern.

Python program:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt



np.random.seed(30)
```

```
months = pd.date_range("2025-01-01", periods=24, freq="MS")

north = 500 + np.random.normal(0, 20, 24)

south = 550 + np.random.normal(0, 25, 24)

east = 450 + np.random.normal(0, 15, 24)

west = 600 + np.random.normal(0, 60, 24)

plt.figure(figsize=(10,5))

plt.plot(months, north, label="North")

plt.plot(months, south, label="South")

plt.plot(months, east, label="East")

plt.plot(months, west, label="West")

plt.xlabel("Month")

plt.ylabel("Electricity Consumption")

plt.title("Monthly Electricity Consumption")

plt.legend()

plt.show()
```

Output:



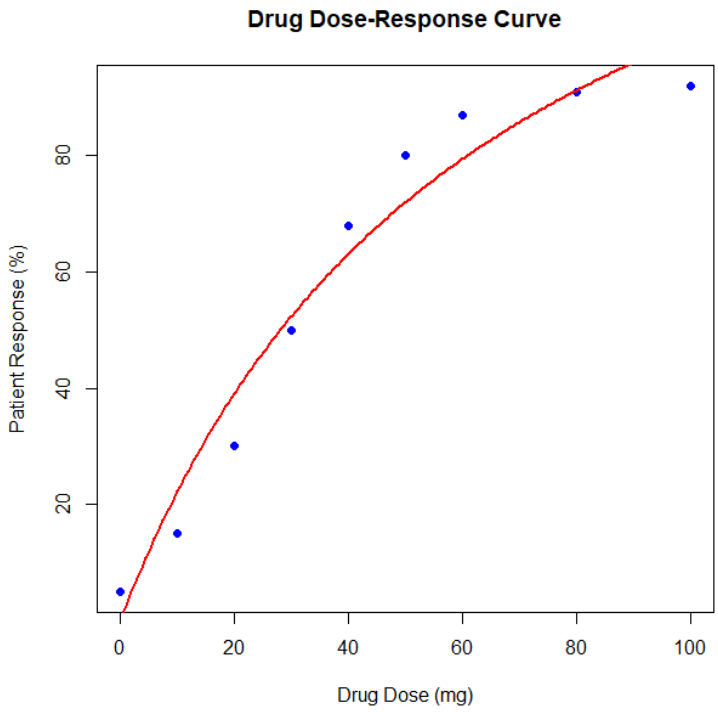
5. A pharmaceutical company is testing a new drug and needs to determine the minimum effective dose before advancing to clinical trials. Using **R**, plot a dose–response curve from the trial data (dose vs. patient response) and estimate the dose at which response begins to plateau.

R program:

```
dose <- c(0, 10, 20, 30, 40, 50, 60, 80, 100)

response <- c(5, 15, 30, 50, 68, 80, 87, 91, 92)



plot(dose, response,

    pch = 19,

    col = "blue",

    main = "Drug Dose-Response Curve",

    xlab = "Drug Dose (mg)",
```

```
    ylab = "Patient Response (%)")


model <- nls(response ~ a * dose / (b + dose),
             start = list(a = 100, b = 30))


dose_seq <- seq(0, 100, length.out = 200)
predicted <- predict(model,
                     newdata = data.frame(dose = dose_seq))


lines(dose_seq, predicted, col = "red", lwd = 2)
```

Output:



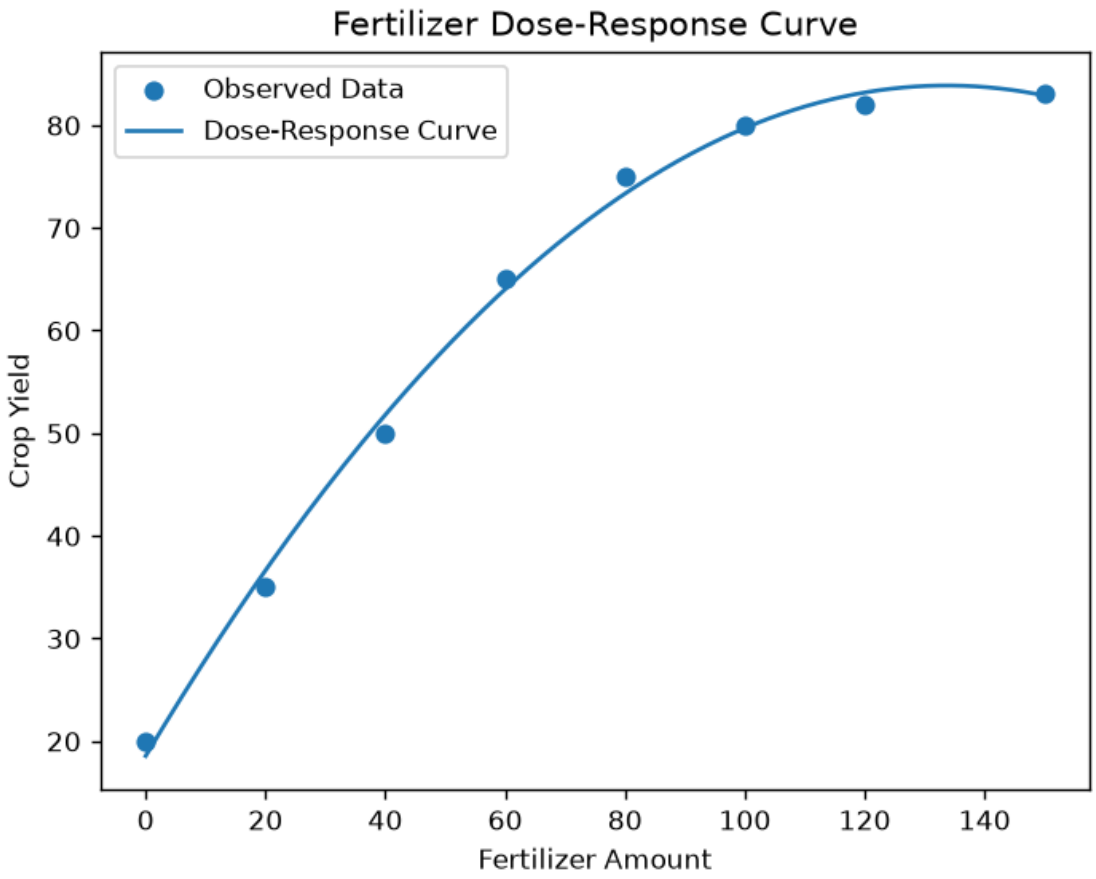
6. An agricultural research lab is studying how increasing fertilizer concentration affects crop yield, aiming to avoid wasteful over-application. Using **Python**, fit and plot a dose–response curve of fertilizer amount vs. yield, and advise the optimal dosage range.

Python program:

```
import numpy as np

import matplotlib.pyplot as plt



fertilizer = np.array([0, 20, 40, 60, 80, 100, 120, 150])

yield_value = np.array([20, 35, 50, 65, 75, 80, 82, 83])



# Polynomial fit

model = np.polyfit(fertilizer, yield_value, 2)

curve = np.poly1d(model)



x = np.linspace(0, 150, 300)



plt.scatter(fertilizer, yield_value, label="Observed Data")

plt.plot(x, curve(x), label="Dose-Response Curve")



plt.xlabel("Fertilizer Amount")

plt.ylabel("Crop Yield")
```

```
plt.title("Fertilizer Dose-Response Curve")

plt.legend()

plt.show()
```

Output:



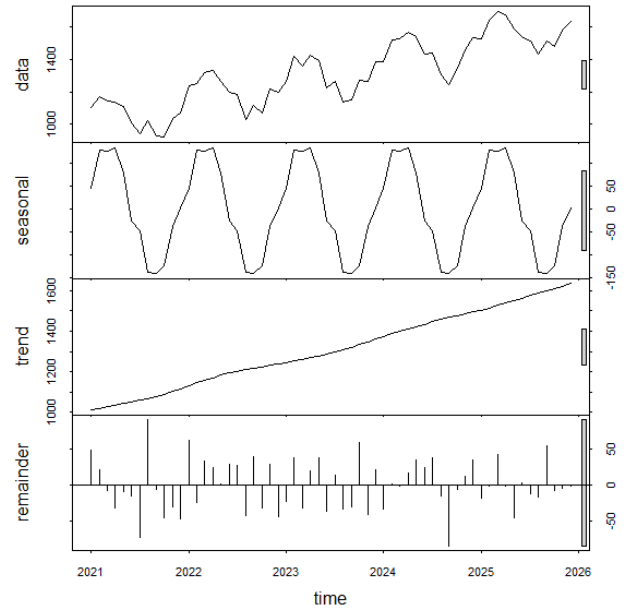
7. An airline's revenue team suspects that ticket sales follow a strong seasonal pattern tied to holidays, but a slow underlying decline is being masked by this seasonality. Using **R**, apply STL decomposition to monthly ticket sales and separate the trend, seasonal, and residual components.

R program:

```
set.seed(40)
```

```
sales <- ts(
  1000 +
  10 * (1:60) +
  150 * sin(2*pi*(1:60)/12) +
  rnorm(60, 0, 40),
  frequency = 12,
  start = c(2021, 1)
)

decomp <- stl(sales, s.window = "periodic")

plot(decomp)
```

Output:



8. A streaming platform wants to know whether recent dips in weekly active users reflect a real decline or just normal seasonal viewing habits. Using **Python**, perform STL decomposition on the weekly user-activity data and interpret the trend component to advise the product team.

Python program:

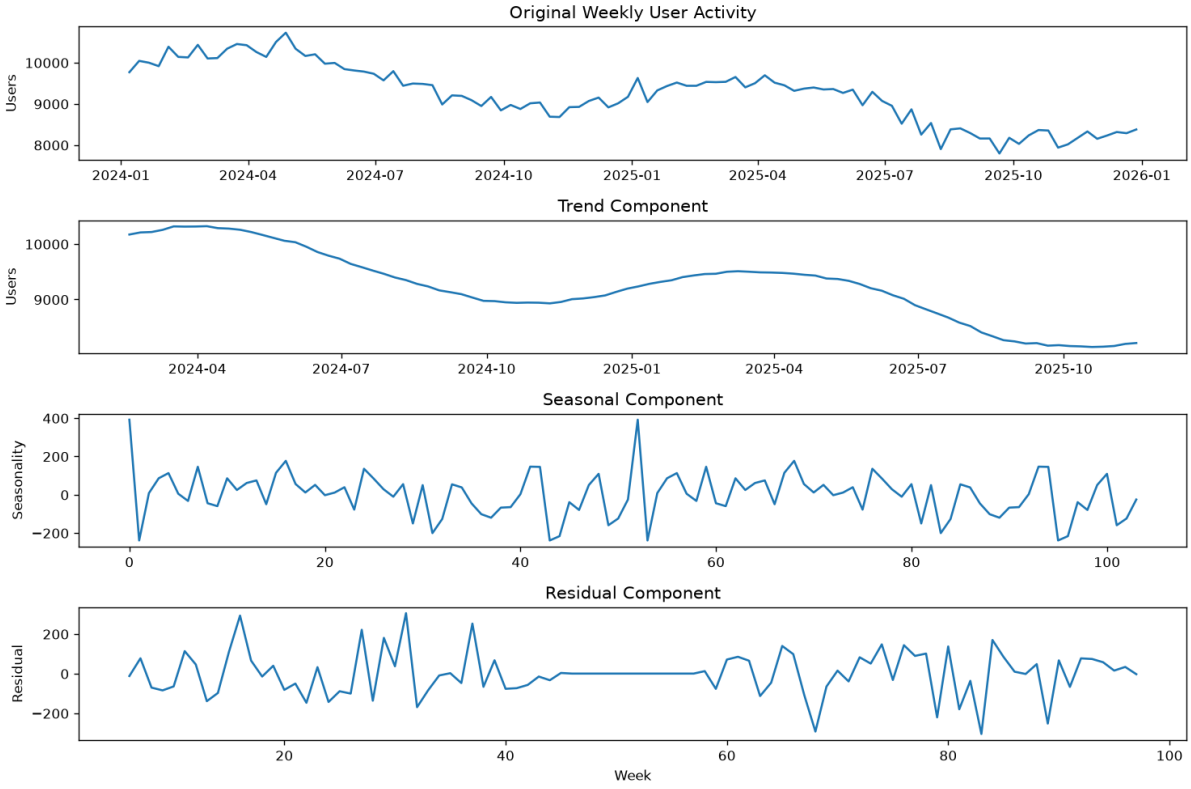
```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt



# Sample weekly user activity data

np.random.seed(50)



weeks = pd.date_range("2024-01-01", periods=104, freq="W")



users = (

    10000

    - np.arange(104) * 15

    + 500 * np.sin(2 * np.pi * np.arange(104) / 52)

    + np.random.normal(0, 150, 104)

)



data = pd.Series(users, index=weeks)
```

```
# ----------------------------

# 1. Calculate Trend

# ----------------------------

trend = data.rolling(window=13, center=True).mean()



# ----------------------------

# 2. Remove Trend

# ----------------------------

detrended = data - trend



# ----------------------------

# 3. Calculate Seasonal Pattern

# ----------------------------

seasonal_pattern = np.zeros(52)



for i in range(52):

    values = detrended.iloc[i::52].dropna()

    seasonal_pattern[i] = values.mean()
```

```
# Repeat seasonal pattern

seasonal = np.tile(seasonal_pattern, 2)



# Match length

seasonal = seasonal[:len(data)]



# ----------------------------

# 4. Calculate Residual

# ----------------------------

residual = data.values - trend.values - seasonal



# ----------------------------

# 5. Plot Components

# ----------------------------

plt.figure(figsize=(12, 8))



plt.subplot(4, 1, 1)

plt.plot(data)

plt.title("Original Weekly User Activity")

plt.ylabel("Users")
```

```
plt.subplot(4, 1, 2)

plt.plot(trend)

plt.title("Trend Component")

plt.ylabel("Users")


plt.subplot(4, 1, 3)

plt.plot(seasonal)

plt.title("Seasonal Component")

plt.ylabel("Seasonality")


plt.subplot(4, 1, 4)

plt.plot(residual)

plt.title("Residual Component")

plt.ylabel("Residual")

plt.xlabel("Week")


plt.tight_layout()

plt.show()
```

Output:



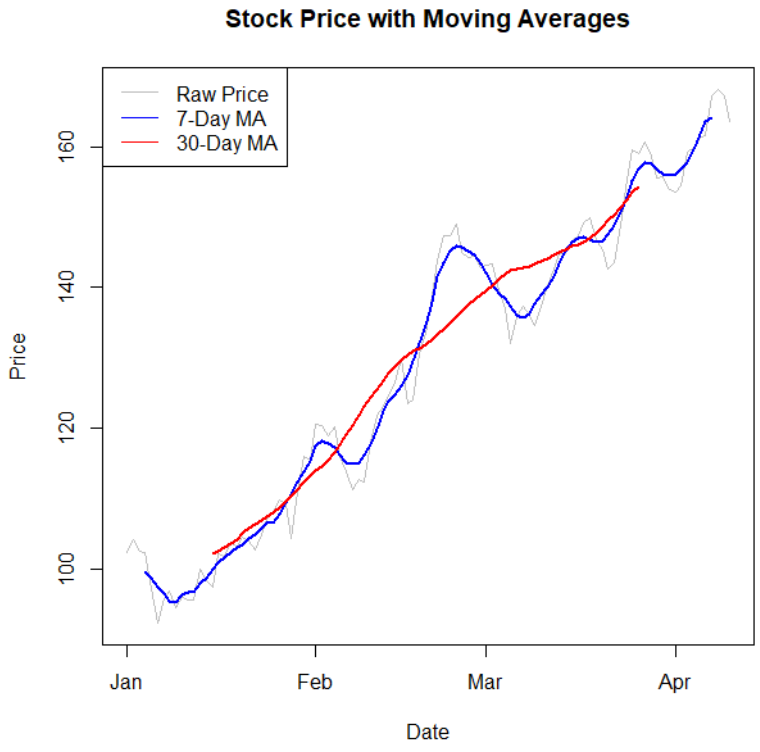
9. A stock trading firm wants to reduce short-term noise in a volatile stock's price data to spot the underlying momentum before making buy/sell calls. Using **R**, compute and plot a 7-day and 30-day moving average over the raw price series, and explain what the crossover between them suggests.

R program:

```
set.seed(60)


dates <- seq(as.Date("2025-01-01"), by = "day", length.out = 100)

price <- 100 + cumsum(rnorm(100, 0.2, 3))


ma7 <- stats::filter(price, rep(1/7, 7), sides = 2)

ma30 <- stats::filter(price, rep(1/30, 30), sides = 2)
```

```
plot(dates, price,
     type = "l",
     col = "gray",
     main = "Stock Price with Moving Averages",
     xlab = "Date",
     ylab = "Price")

lines(dates, ma7, col = "blue", lwd = 2)
lines(dates, ma30, col = "red", lwd = 2)

legend("topleft",
       legend = c("Raw Price", "7-Day MA", "30-Day MA"),
       col = c("gray", "blue", "red"),
       lty = 1)
```

Output:



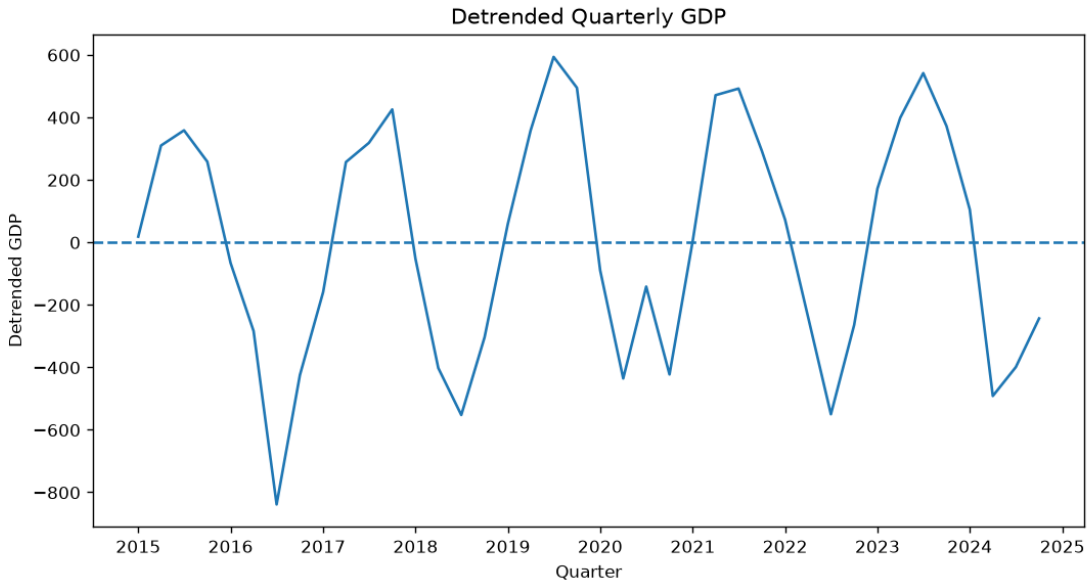
10. An economist is analyzing a country's quarterly GDP data but needs to remove the long-term growth trend to study business-cycle fluctuations in isolation. Using **Python**, detrend the GDP series (e.g., by differencing or regression-based detrending), plot the detrended series, and interpret the remaining fluctuations.

Python program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Create quarterly dates
quarters = pd.date_range("2015-01-01", periods=40, freq="QS")

# Sample GDP data
np.random.seed(70)

gdp = (
    20000
    + np.arange(40) * 150
    + 500 * np.sin(2 * np.pi * np.arange(40) / 8)
    + np.random.normal(0, 100, 40)
)

# Create time index
x = np.arange(len(gdp))

# Calculate trend using NumPy polynomial fitting
trend_coefficients = np.polyfit(x, gdp, 1)
```

```
# Calculate trend values
trend = np.polyval(trend_coefficients, x)

# Remove trend
detrended = gdp - trend

# Plot detrended GDP
plt.figure(figsize=(10, 5))

plt.plot(quarters, detrended)

plt.axhline(0, linestyle="--")

plt.xlabel("Quarter")
plt.ylabel("Detrended GDP")
plt.title("Detrended Quarterly GDP")

plt.show()
```

Output:


Detrended Quarterly GDP
Detrended GDP
600
400
200
0
−200
−400
−600
−800
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
Quarter